# Colinha de comandos

Detector de voz sintética · TCC — Ciência da Computação, UNIP · todos os comandos são para o PowerShell, rodados de dentro da pasta `ml\`

## Como o sistema funciona, em uma linha

Áudio: arquivo, som do computador ou microfone → Conversão para 16 kHz (filtro FIR) → Janelas de 4 s, a cada 2 s → Modelo dá um score de 0 a 1 por janela → Média ponderada pela fala → Comparação com o limiar

- **Score:** 0 = voz humana, 1 = voz sintética. É um **indício**, nunca prova.
- **Modelo padrão:** `baseline_v2` (LFCC + CNN), treinado na ASVspoof 2019 LA.
- **Limiar escolhido sozinho pelo tipo de áudio:** 0,654 para arquivo nativo de 16 kHz (o formato da base); 0,686 (aparece como 0,69) para captura ao vivo e arquivos gravados a 44,1/48 kHz, que passam pela conversão de taxa.
- **Peso da janela:** janela com pouca fala (o resto é silêncio) pesa menos na média, porque o modelo a completa por repetição e ela sai do padrão do treino.

## 1. Preparar o ambiente

```
git pull
```

**O que faz:** baixa as últimas mudanças da branch para o seu computador. Rode antes de testar, para não usar uma versão antiga.

```
cd ml
pip install -r requirements.txt -r web\requirements.txt
```

**O que faz:** entra na pasta do código e instala as bibliotecas do modelo e da interface (FastAPI, Uvicorn, Jinja2). Só na primeira vez ou quando o arquivo de requisitos mudar. Para captura ao vivo, também é preciso o `soundcard` (`pip install soundcard`).

## 2. Interface web

```
uvicorn web.app:app
```

**O que faz:** sobe o servidor da interface. Abra `http://127.0.0.1:8000` no navegador. Para parar: **Ctrl+C** no terminal.

```
uvicorn web.app:app --reload
```

**O que faz:** igual ao anterior, mas reinicia sozinho quando um arquivo do código é salvo. Útil para desenvolver; na apresentação, use o comando sem `--reload`. Rode **só um** dos dois.

### As abas

| Aba | O que faz |
|---|---|
| Enviar arquivo | Analisa um áudio (WAV, MP3, M4A, FLAC, OGG, Opus, AAC, até 25 MB). O arquivo é apagado logo depois da análise; só o resultado fica guardado. |
| Resultado | Score médio, limiar usado, decisão e um gráfico com uma barra por janela (verde abaixo do limiar, vermelha acima, apagada = pouca fala). Botão para baixar o relatório em JSON. |
| Ao vivo | Escolhe a fonte (**som do computador** ou **microfone**) e o dispositivo, inicia e para a captura. Ao parar, a sessão vai para o Histórico. |
| Clonar | Escolhe a voz (só as autorizadas por termo) e a ferramenta (XTTS ou Chatterbox), digita a frase e gera o clone; ele toca na página e já sai analisado. Leva de 1 a 2 minutos. Pare o ao vivo antes: a GPU não comporta os dois. |
| Conferir | No cabeçalho, ao lado de Sobre: roda sozinha quando o servidor abre e mostra "Pronto", "Atenção" ou "Falha". A página lista o que conferiu (modelo, limiar, GPU, microfone, som, clonagem, disco) e o que fazer. Pelo terminal: `python scripts\conferir.py`. |
| Histórico | Lista as análises. Excluir uma sessão ao vivo apaga também a gravação dela; "Limpar todo o histórico" apaga tudo. |

### Trocar o modelo (opcional)

```
$env:DETECTOR_CONFIG = "configs\baseline_v2.yaml"
$env:DETECTOR_CHECKPOINT = "checkpoints\baseline_lfcc_cnn_v2.pt"
uvicorn web.app:app
```

**O que faz:** define qual modelo o servidor carrega (os valores acima já são o padrão). As variáveis valem só para aquele terminal. Outras: `DETECTOR_DEVICE` (cpu ou cuda), `DETECTOR_BANCO` (arquivo do histórico), `DETECTOR_PASTA_AO_VIVO` (onde ficam as gravações).

### Modelo final: XLS-R congelado

```
$env:DETECTOR_XLSR = "outputs\limiar_xlsr\limiar.json"
$env:DETECTOR_DEVICE = "cpu"
uvicorn web.app:app
```

**O que faz:** sobe a interface com o modelo final. O JSON traz o limiar, a cabeça treinada e a pasta do XLS-R; o modelo carrega na primeira análise. Com `cpu`, o Enviar arquivo leva uns 3 s por áudio e deixa a GPU para o ao vivo e para a aba Clonar (o Chatterbox ocupa quase toda ela). As duas abas analisam só a fala, **frase por frase**, com o mesmo limiar: no Enviar arquivo, o score é a média das frases; no Ao vivo, o resultado aparece cerca de 1 s depois que a pessoa para de falar.

```
python monitor.py --xlsr outputs\limiar_xlsr\limiar.json --fonte microfone
```

**O que faz:** o mesmo ao vivo no terminal, uma linha por frase. Uma pausa de 0,8 s fecha a frase.

```
python scripts\gravar_frases.py voz --frases docs\etica\frases_ensaio5.txt --piso-minimo 15 --saida
data\vozes\calibracao\L03
```

**O que faz:** grava as vozes de calibração, 12 s por frase, com as checagens depois de cada uma. Só com o termo de consentimento assinado antes.

```
python scripts\limiar_xlsr.py --real L02=data\vozes\calibracao\L02 --real L03=data\vozes\calibracao\L03
```

**O que faz:** refaz o JSON do limiar com as vozes reais: o limiar é o maior score das frases delas, e vale para as duas abas. Reinicie o servidor depois.

## 3. Monitor no terminal (sem a página)

É o mesmo analisador da aba Ao vivo, mas mostrando cada janela numa linha. Bom para explicar o funcionamento passo a passo.

```
python monitor.py --listar-dispositivos
```

**O que faz:** mostra os nomes exatos das saídas de som e dos microfones, com o padrão do Windows marcado. Use o nome em `--dispositivo-audio`.

```
python monitor.py --config configs\baseline_v2.yaml --checkpoint checkpoints\baseline_lfcc_cnn_v2.pt
```

**O que faz:** analisa ao vivo o **som do computador** (o que sai no alto-falante: chamada, vídeo, player). Para encerrar: **Ctrl+C**; no fim aparece o resumo.

```
python monitor.py --config configs\baseline_v2.yaml --checkpoint checkpoints\baseline_lfcc_cnn_v2.pt --
fonte microfone
```

**O que faz:** analisa ao vivo o **microfone padrão** do Windows.

```
python monitor.py --config configs\baseline_v2.yaml --checkpoint checkpoints\baseline_lfcc_cnn_v2.pt `
    --fonte microfone --dispositivo-audio "Nome do microfone" --gravar teste_mic.wav
```

**O que faz:** usa um microfone específico e salva o que ouviu em `teste_mic.wav`, para analisar de novo depois. A crase no fim da linha continua o comando na linha de baixo (PowerShell).

```
python monitor.py --config configs\baseline_v2.yaml --checkpoint checkpoints\baseline_lfcc_cnn_v2.pt --
arquivo teste_mic.wav
```

**O que faz:** analisa um arquivo com o mesmo caminho do ao vivo. Dá sempre o mesmo resultado para o mesmo arquivo; se ele for da ASVspoof, mostra também o rótulo verdadeiro.

### Outras opções úteis

| Opção | O que faz |
|---|---|
| `--segundos 30` | Encerra sozinho depois de 30 s. |
| `--json saida.json` | Grava os scores de cada janela num arquivo (é o que a página lê). |
| `--hop 1` | Passo entre janelas em segundos (padrão: 2). |
| `--device cuda` | Usa a GPU, se houver (padrão: o do config). |

### Como ler a tabela do monitor

| Coluna | Significado |
|---|---|
| t | Instante de início da janela, em segundos. |
| score | Probabilidade de síntese daquela janela (0 a 1). |
| média | Média ponderada das janelas mais recentes: o valor que decide. |
| canal | Quanto da energia está acima de 4 kHz. Perto de 0% por várias janelas indica áudio de banda estreita (telefone), e essas janelas ficam fora da média. |
| peso | Fração de fala na janela (1,00 = cheia). Abaixo de 0,95 aparece "janela parcial". |
| sinal | Barra do score; o `\|` marca o limiar. "(silêncio)" = nada captado. |

## 4. Demonstração com áudios da base

```
python scripts\montar_demo.py --config configs\baseline_v2.yaml
```

**O que faz:** monta, em `outputs\demo\`, três playlists com áudios do conjunto de avaliação da ASVspoof 2019: `reais.wav` (3 vozes humanas), `falsos.wav` (3 sintéticas, ataques A09, A11 e A19) e `misto.wav` (alternados). Cada uma vem com uma "cola" `.txt` com o rótulo e o score esperado de cada trecho.

```
python scripts\canal_real.py tocar --arquivo outputs\demo\reais.wav
python scripts\canal_real.py tocar --arquivo outputs\demo\falsos.wav
```

**O que faz:** toca a playlist no alto-falante sem alterar o áudio. Ele espera um **Enter** para começar: inicie a análise ao vivo antes, espere ela carregar e só então aperte Enter. Alternativa: o Reprodutor de Mídia do Windows. **Não use o VLC**: ele altera o áudio (o mesmo arquivo de vozes reais foi de 0,33 para 0,98).

| Playlist | Enviada como arquivo | Ao vivo, som do computador | Pelo celular → microfone |
|---|---|---|---|
| reais | 0,03 | 0,33 — sem indício | 0,07 a 0,19 |
| falsos | 1,00 | 1,00 — indício | 0,15 a 0,35 (não separa) |

## 5. Testes com a própria voz

```
python scripts\gerar_sintetico.py --entrada gravacoes\ --saida outputs\demo_propria
```

**O que faz:** pega suas gravações e refaz cada uma com dois vocoders usados nos ataques da base: Griffin-Lim (ataque A11) e WORLD (A02, A03, A05, A07). Original e falso têm a mesma voz, microfone e frase; só muda o vocoder. Gera também `pareado.wav`, com tudo em sequência. O WORLD precisa de `pip install pyworld "setuptools<81"`.

```
python scripts\avaliar_arquivos.py outputs\demo_propria
python scripts\avaliar_arquivos.py teste_mic.wav outputs\demo\reais.wav
```

**O que faz:** mostra o score de cada arquivo numa tabela, pelo mesmo caminho da página. Numa pasta do `gerar_sintetico`, compara original, Griffin-Lim e WORLD de cada frase.

## 6. Conferir que está tudo certo

```
python -m pytest tests -q
```

**O que faz:** roda os testes automatizados (cerca de 680). No fim deve aparecer "passed" sem "failed". O GitHub roda os mesmos testes a cada envio.

## 7. Antes de testar ao vivo

- **Desligue os aprimoramentos de áudio** da saída e do microfone: Configurações › Som › (dispositivo) › Aprimoramentos de áudio. Na saída, eles levam todo score para perto de 1,0; no microfone, sobem o score da voz humana.
- Toque os áudios pelo Reprodutor do Windows ou pelo `canal_real.py tocar`, nunca pelo VLC.
- Ensaie na mesma sala e com o mesmo microfone da apresentação.

### Problemas comuns

| Sintoma | Causa e solução |
|---|---|
| Tudo aparece como "(silêncio)" no som do computador | Nada estava tocando, ou o som sai por outro dispositivo (fone). Rode `--listar-dispositivos` e escolha a saída certa. |
| Tudo em silêncio no microfone | Microfone mudo, sem permissão no Windows (Privacidade › Microfone) ou dispositivo errado. |
| Todos os scores perto de 1,0, até de voz humana | Aprimoramentos de áudio ligados, ou o áudio passou pelo VLC. |
| "Não foi possível ler o áudio" com M4A ou AAC | Esses formatos precisam do FFmpeg (`winget install Gyan.FFmpeg` e reabrir o terminal). Na demonstração, use WAV. |
| "Já há uma sessão ao vivo em andamento" | Uma sessão ficou aberta: clique em "Parar análise" ou reinicie o servidor. |
| Aviso de "amostras perdidas" no fim | O computador ficou sobrecarregado. Feche outros programas de áudio e grave de novo. |
| Erro ao iniciar o servidor (porta em uso) | Já há um servidor rodando em outro terminal. Feche-o com Ctrl+C ou use `uvicorn web.app:app --port 8001`. |

### Respostas rápidas para a banca

| Pergunta | Resposta curta |
|---|---|
| Por que um filtro FIR na conversão? | O conversor padrão apagava a faixa de 7,6–8 kHz, que o modelo usa: a captura custava 5,68 pontos de EER. Com o FIR, custa 0,39 (19,41% contra 19,02%). |
| Por que dois limiares? | Áudio que passa por 44,1/48 kHz chega diferente ao modelo. O segundo limiar foi calibrado na partição de desenvolvimento com a mesma conversão, sem tocar no conjunto de avaliação. |
| Por que a página não grava pelo navegador? | O navegador não tem acesso ao som do sistema. A página inicia o `monitor.py` como processo e para ele criando um arquivo que o monitor vigia. |
| Funciona com vozes de IA atuais? | Não de forma confiável. O modelo conhece ataques de 2018–2019 e, com vozes fora da base, reage às condições de gravação. Isso foi medido e está no trabalho. |
| O que fica guardado? | Só os resultados, num banco SQLite local. O arquivo enviado é apagado; a gravação ao vivo fica no computador até a sessão ser excluída. |
| Por que o ao vivo do modelo final espera a pausa? | O limiar foi calibrado com frases inteiras, e a média de pedaços de 4 s não dá o mesmo número: a diferença chega a 0,67 nos log-odds. |
| Por que dois limiares no modelo final? | A gravação de 12 s leva uns 5 s de ruído da sala, e a frase do ao vivo não. Cada caminho tem o seu limiar, tirado das mesmas vozes reais. |

Números de RESUMO_TCC.md (seções 10.2.1 a 10.2.7; modelo final, 10.2.43 e 10.2.44). Modelo padrão: baseline_v2; o modelo final entra com `DETECTOR_XLSR`.